RICE UNIVERSITY

Pipeline and 3D Parallelism

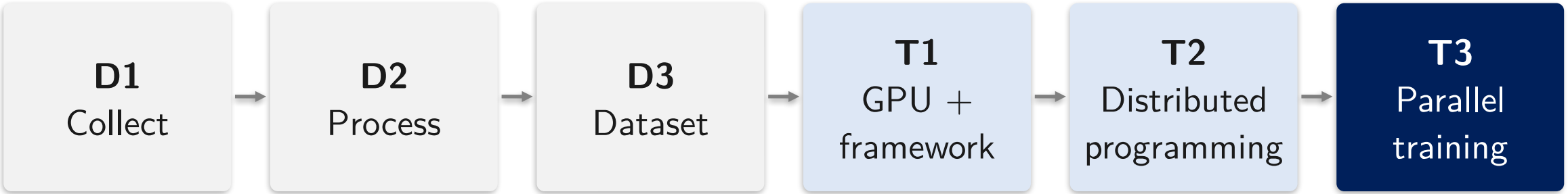
Cutting a model into stages, filling the pipeline, and choosing a 3D layout

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 13, Lecture 1

Where we are in the pipeline



Week 12: split the batch (data parallelism), split each layer (tensor parallelism)

In our pipeline

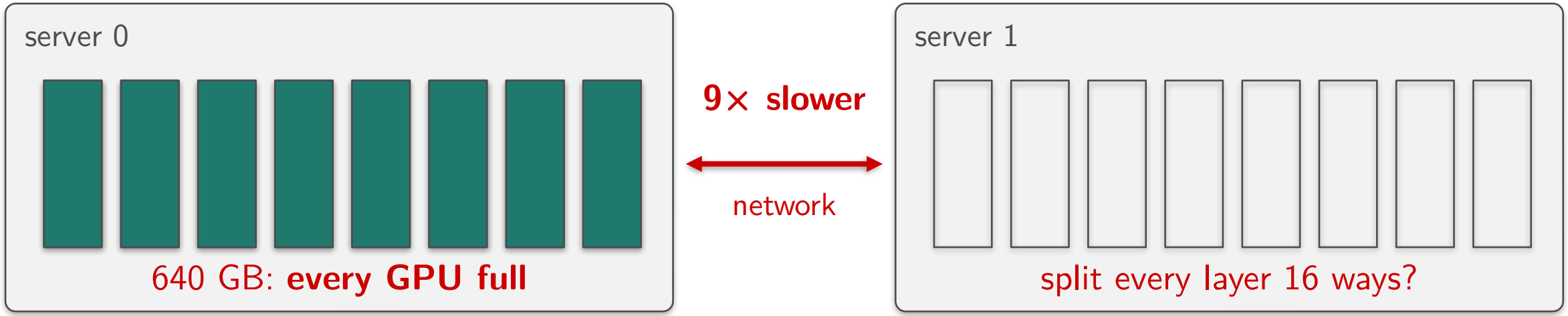
Today: split the model **by layers** across GPUs, then combine data $\times$ tensor $\times$ pipeline into one layout.

After this lecture: **T3 checkpoint** and **Lab 5** are due.

Why do we need pipeline parallelism?

$\Psi = 40 \times 10^9$ parameters, tensor-parallel over one 8-GPU server.

Does it fit?



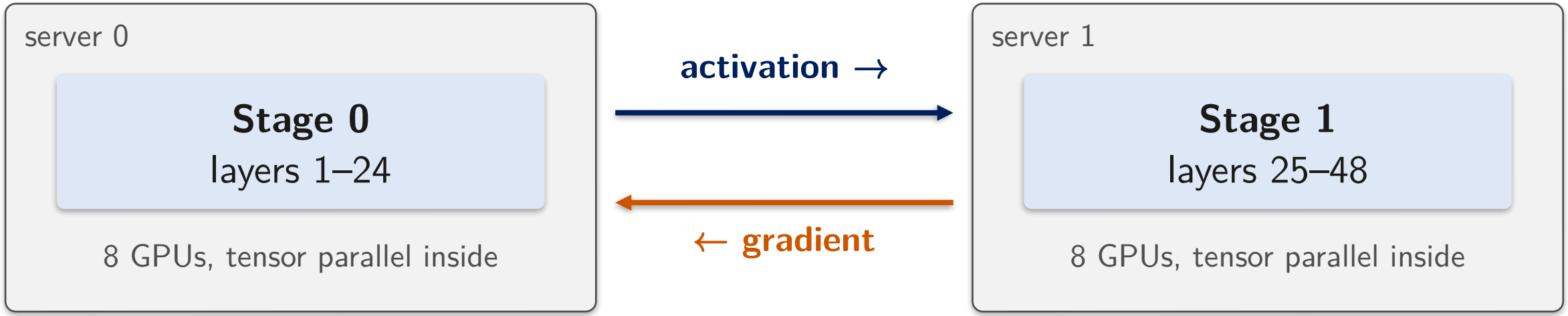
Watch out

Memory is full, and splitting every layer across servers is too chatty. We need a split that **crosses servers rarely**.

Source: Rajbhandari et al., ZeRO, SC 2020 (16 bytes per parameter); H100 and DGX H100 datasheets.

Pipeline parallelism, in plain words

A **stage** is a group of consecutive layers on its own GPUs. Like an assembly line, each stage does its part and **passes the result on**.



Key idea

Each GPU holds **40 GB**, and only the result at the cut crosses the slow network.

Where pipeline parallelism shows up

- **GPipe** (Google) and **PipeDream** (Microsoft), 2019: the schedules of Part 2.
- **Megatron-LM** adds tensor and data parallelism: 10^{12} parameters on 3,072 GPUs.
- **Cost**: idle time while the line fills and empties, plus stored activations.
- **In this course**: Lab 5 (stages, bubbles); T3's layout simulator.

In our pipeline

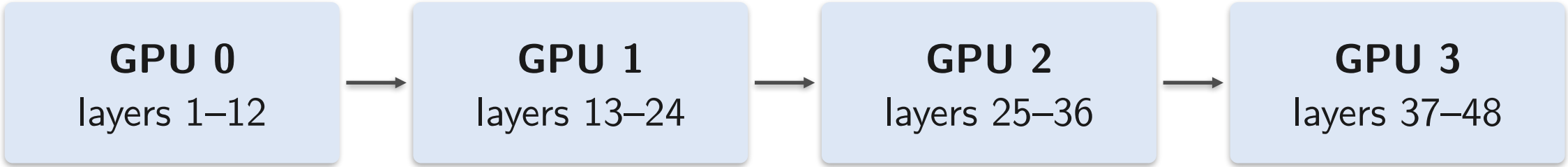
Why pipeline parallelism? When a model outgrows one server, cut it **between layers** so only small messages cross the slow network.

Source: Huang et al., "GPipe," NeurIPS 2019; Narayanan et al., "PipeDream," SOSP 2019; Narayanan et al., SC 2021.

Learning goals for today

- **Cut** a model into pipeline stages that do equal work, and say what crosses each cut.
- **Derive** the bubble fraction $\mathbf{(p-1)/(m+p-1)}$ from a timeline.
- **Compare** the GPipe and 1F1B schedules: same bubble, different memory.
- **Choose** a d $\times$ t $\times$ p layout: where each group goes, and memory per GPU $\approx 16\Psi/(t \cdot p)$.

A puzzle to start



A 48-layer model is split across 4 GPUs, 12 layers each. One batch goes through forward, then backward. What fraction of the time is each GPU busy?

Answer

Only **25%**: layer 13 must wait for layer 12, so one GPU works at a time. Split the batch into 8 pieces and it rises to $\mathbf{8/11 \approx 73\%}$.

Part 1

Cutting the model into stages

Where do we cut the model, and what crosses each cut?

Three ways to split the work

Split	What is split	When GPUs talk	Price
Data (d)	the batch	once per step	full copy per GPU
Tensor (t)	each weight matrix	inside every layer	needs fast links
Pipeline (p)	the layers	at each cut	idle time

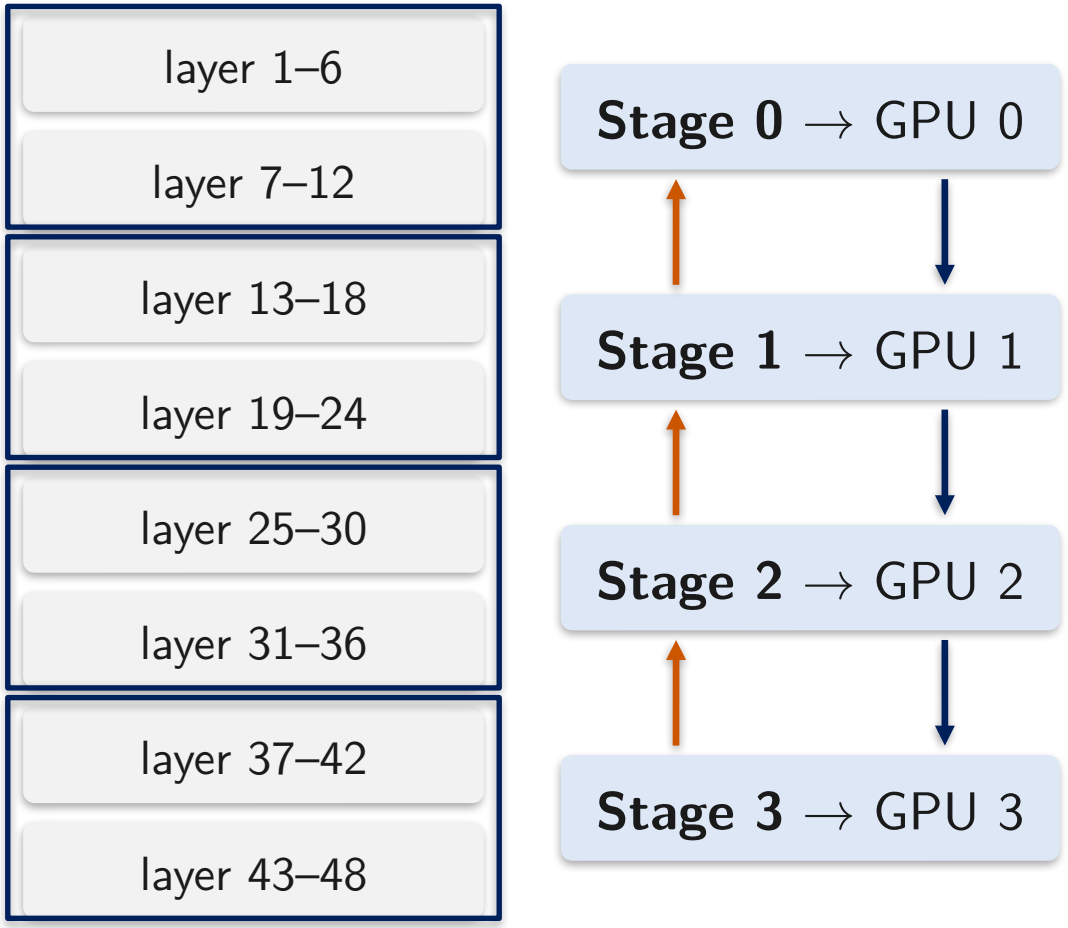
d, t, p = how many GPUs share each kind of split

Key idea

Pipeline parallelism stores **1/p of the layers** per GPU and sends only what crosses a cut.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021.

Layer stages



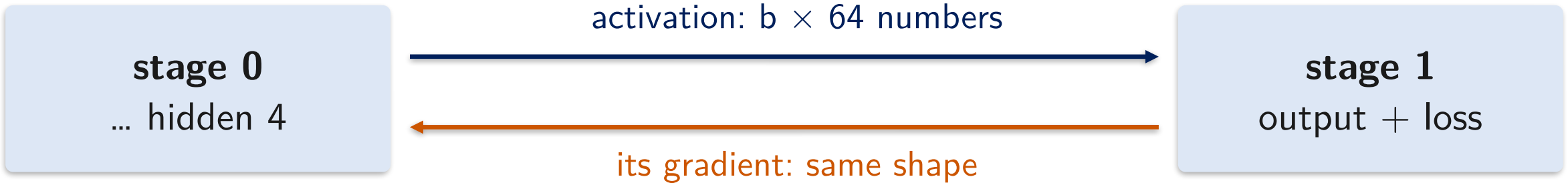
Forward: send the activation (the stage's output) to the next GPU

Backward: send the gradient of the loss w.r.t. that activation back

Key idea

Each GPU holds **1/p of the layers** and their optimizer state.

What crosses a cut



$$b = 16: \;\; 16 \times 64 \times 4 \text{ bytes} = 4 \text{ KB per message}$$

Memory

Two small messages per cut, not traffic inside every layer: a pipeline can cross the **slower links between servers**.

A cut in code

```
# stage s: compute and send forward
h = stage(x)
dist.send(h.detach(), dst=rank + 1)

# stage s + 1: receive into a buffer
h = torch.empty(b, 64)
dist.recv(h, src=rank - 1)
h.requires_grad_()      # so h.grad exists
```

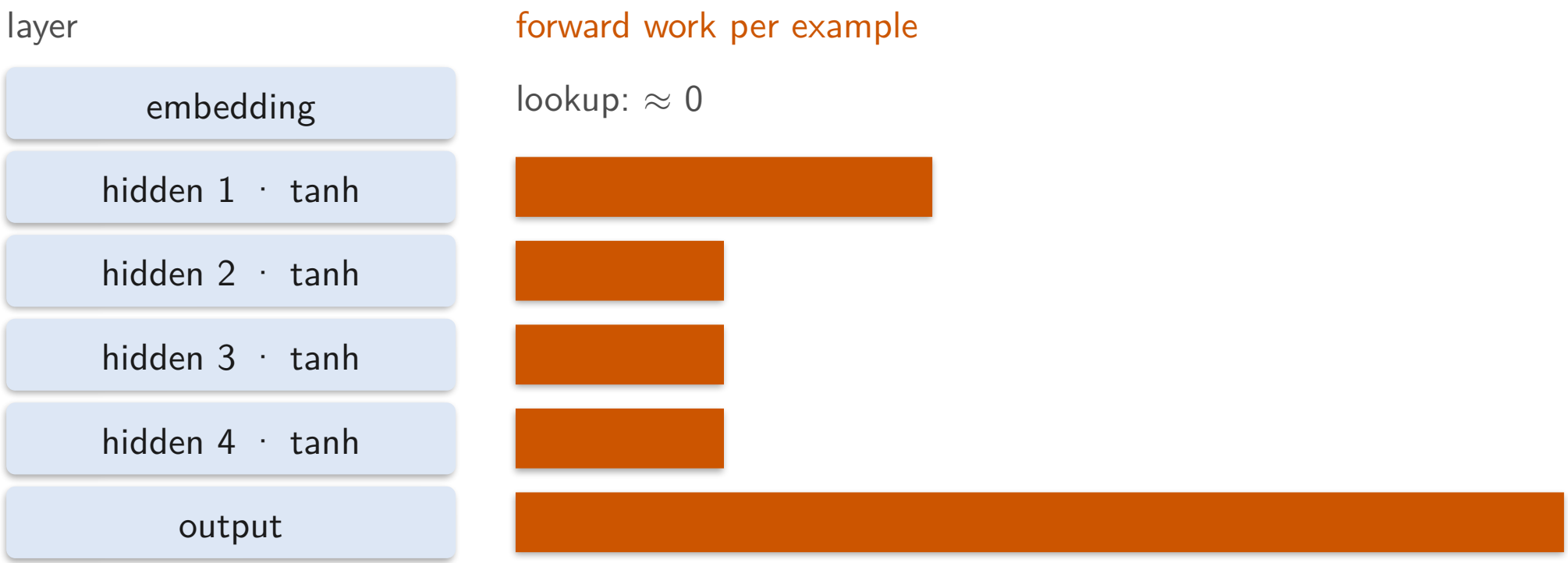
- Each **send** needs a **recv**.
- The receiver must know the **shape**.

Key idea

Backward is the mirror image: the receiver sends **h.grad** back to the sender.

Source: PyTorch documentation, torch.distributed: send, recv.

Our deeper toy model



Compute

46,818 parameters. The output layer alone does **half** of the work.

Balance the work, not the layer count

cut by layer count

stage 0: embed + hidden 1–2	stage 1: hidden 3–4 + output

stage 1 does **2.3×** the work of stage 0

cut by work

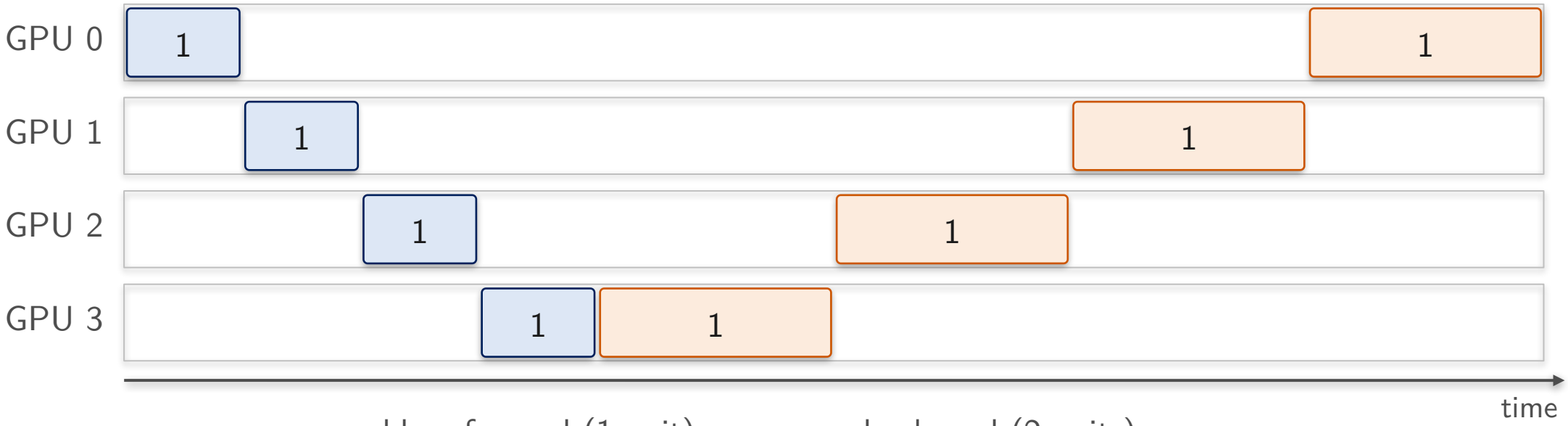
stage 0: embed + hidden 1–4	stage 1: output

the two stages do **almost equal** work

Watch out

A pipeline runs at the pace of its **slowest stage**: balance work, not layer counts.

The naive schedule, one step at a time



blue: forward (1 unit) · orange: backward (2 units)

Watch out

At every moment **exactly one** GPU is busy: a slower single GPU with more memory.

Naive pipeline: utilization

tF, tB: time of one stage's forward, backward

$$\text{step time} = p \cdot (\text{tF} + \text{tB})$$

$$\text{busy time per GPU} = \text{tF} + \text{tB}$$

$$\text{utilization} = 1 \,/\, p, \quad \text{idle} = (p-1) \,/\, p$$

Watch out

p = 4: each GPU is busy only **25%** of the time. Naive pipelining fixes **memory**, not **speed**.

Check yourself

Two minutes with a neighbour.

- A piece of 32 examples crosses a cut after a width-64 layer, in FP32. How many bytes?
- Can our deeper MLP be balanced across **3** stages? Why or why not?
- Naive pipeline over 8 GPUs: what fraction of the time is each GPU busy?

Part 2

Microbatches and the bubble

How do we keep every stage busy?

Split the batch into microbatches

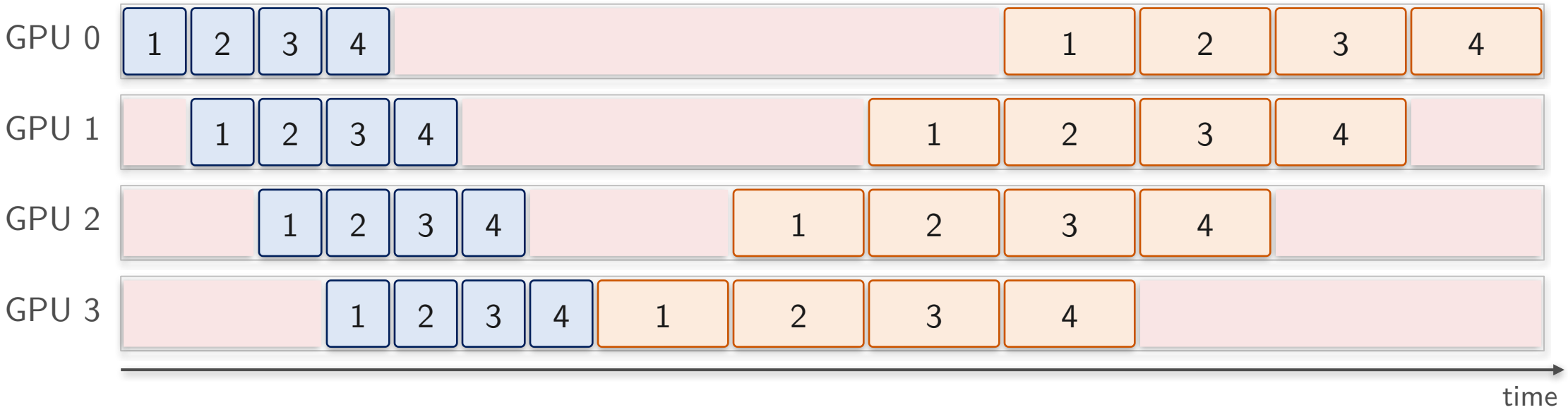
$$B = m \cdot b$$

$$\nabla L(\text{batch}) = (1/m)\ \Sigma\ \nabla L(\text{microbatch } j)$$

- **microbatch**: one of the m equal pieces of the batch.
- Run microbatches through the stages like an **assembly line**.
- **Add up** gradients; one optimizer step after the last one.

Source: Huang et al., "GPipe," NeurIPS 2019; Narayanan et al., SC 2021, §2.2.

The GPipe schedule: p = 4, m = 4



red: bubble (idle)

Key idea

Each GPU is busy **12 of 21** units: the bubble is **9/21 ≈ 43%**.

Deriving the bubble fraction

$$\text{step time} = (m + p - 1) \cdot (\text{tF} + \text{tB})$$

$$\text{busy time} = m \cdot (\text{tF} + \text{tB})$$

$$\text{bubble fraction} = (p - 1) \;/\; (m + p - 1)$$

Result

m $= 1$ gives the naive (p $-$ 1)/p. As m grows the bubble vanishes: we need **m** $\gg$ **p**.

Source: Huang et al., "GPipe," NeurIPS 2019; Narayanan et al., SC 2021, §2.2.

How many microbatches are enough?

p = 4 stages	**m = 4**	**m = 8**	**m = 16**
bubble	43%	27%	16%

Key idea

GPipe's rule of thumb: $\mathbf{m \geq 4p}$ microbatches. Deeper pipelines need more of them.

Source: Huang et al., "GPipe," NeurIPS 2019; Narayanan et al., SC 2021, §2.2.

Microbatches are not free

$$B = m \cdot b \quad (B \text{ is fixed by the optimizer})$$

- Smaller b $\rightarrow$ **smaller matrix products** $\rightarrow$ slower per example (Week 8).
- More messages $\rightarrow$ each pays the **latency** α (Week 11).
- Fixed **cost per microbatch**: launches, Python — large for our MLP.

Watch out

Pick m large enough to shrink the bubble, b large enough to keep the GPU efficient.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021.

GPipe's hidden cost: activation memory

- **activations**: layer outputs saved in forward because backward needs them.
- GPipe runs **all m forwards first**: every stage holds m sets at once.
- Small bubble needs large m $\rightarrow$ large memory: the two goals **conflict**.

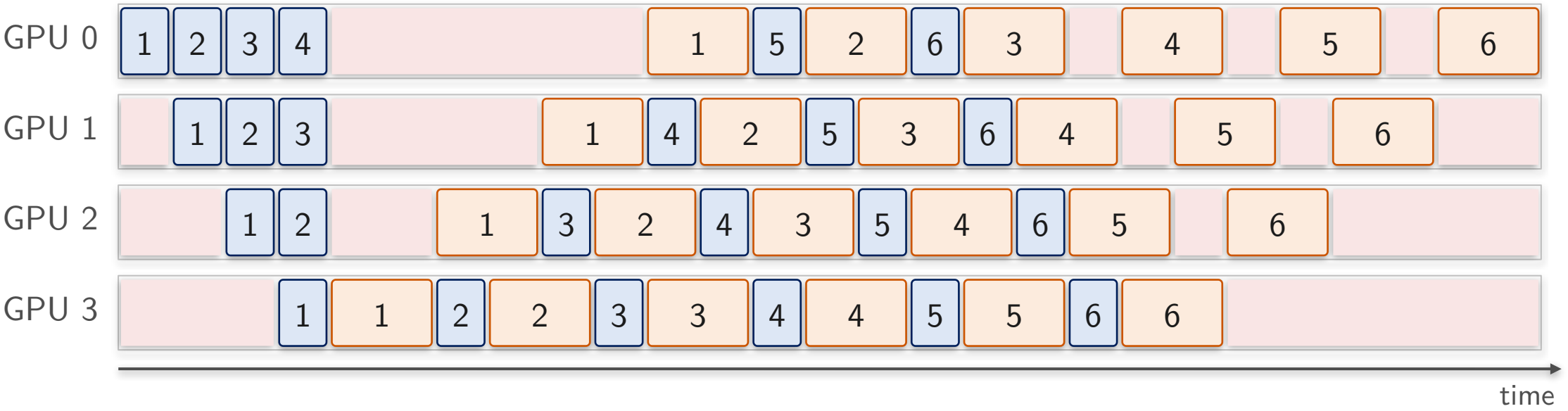
$$\text{stored at the peak} \propto m \text{ microbatches}$$

Watch out

Fixes: **recompute** activations in backward, or **reorder** the schedule (1F1B).

Source: Huang et al., "GPipe," NeurIPS 2019; Narayanan et al., SC 2021, §2.2.

The 1F1B schedule: p = 4, m = 6



warm-up → steady: one forward, one backward → cool-down

Key idea

Same step time, same bubble **3/9**; but each microbatch's activations are freed **early**.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. §2.2.1.

1F1B bounds the stored activations

Schedule	Bubble	Stored at once (stage 0)
GPipe	$(p-1)/(m+p-1)$	m microbatches
1F1B	$(p-1)/(m+p-1)$	p microbatches

$p = 4$, $m = 32$: GPipe stores **32**, 1F1B stores **4**

Check

1F1B lets us raise **m** (smaller bubble) **without** raising activation memory.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. §2.2.1.

Check yourself

Two minutes with a neighbour.

- $p = 8$, $m = 16$: what is the bubble fraction? What m brings it under 10%?
- Same p and m: how many microbatches' activations does GPipe store? 1F1B?
- Give one reason not to make m as large as possible.

Part 3

Composing 3D parallelism

Given 64 GPUs, how many go to each kind of split?

N = d × t × p



t = 8 GPUs split each layer

× p = 2 stages = one copy of the model



× d = 4 copies, each on its own part of the batch

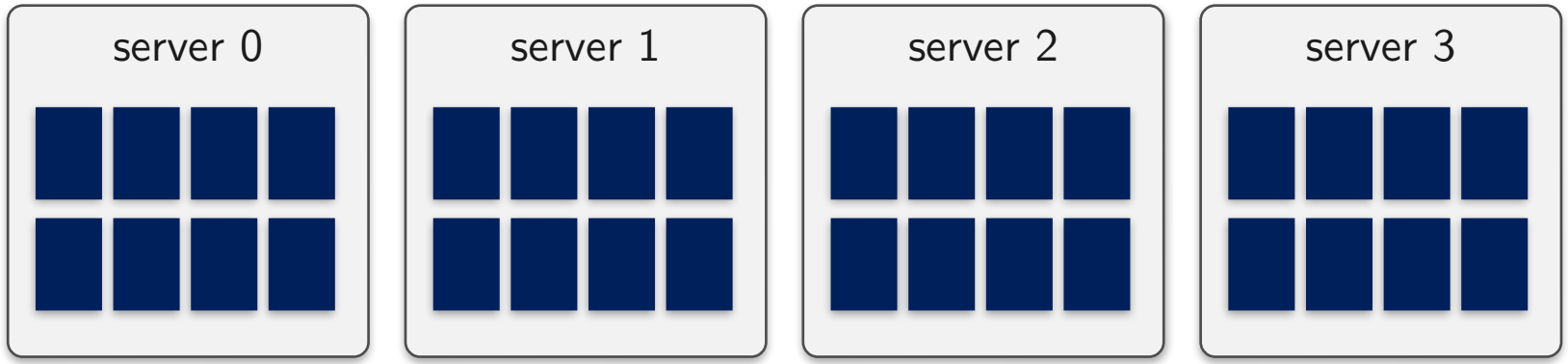
$$N = d \cdot t \cdot p$$

$$64 = 4 \cdot 8 \cdot 2$$

Key idea

Each GPU belongs to **one** group of each kind.

Two kinds of links



... 8 servers of 8 GPUs

inside a server (NVLink): **450 GB/s** per GPU, per direction

between servers (network): **50 GB/s** per GPU, per direction

Memory

Crossing servers is $\approx$ **9× slower**.

Source: H100 datasheet (NVLink 900 GB/s bidirectional); NVIDIA DGX H100 datasheet (8 × 400 Gb/s ConnectX-7 for the compute fabric).

The placement rule

t inside a server	→	talks inside **every layer**: needs the fastest link
p across servers	→	sends **small messages** only at the cuts
d outermost	→	talks **once per step**, overlapped with backward

Key idea

The **chattiest** split goes on the **fastest** link.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. Takeaways #1–2 (paraphrased).

Memory per GPU

$$\text{memory per GPU} \approx 16\Psi \;/\; (t \cdot p) \;\; \text{bytes}$$

Ψ parameters, 16 bytes each (Week 12), spread over the t · p GPUs of one copy

$\boldsymbol{\Psi = 40 \times 10^9}$
one copy: 640 GB



$\mathbf{t \cdot p = 16}$
40 GB per GPU

Memory

Data parallelism does **not** divide it. Leave room for **activations** on top.

Source: Rajbhandari et al., "ZeRO," SC 2020 (16 bytes per parameter for mixed-precision Adam).

Worked layout: 64 GPUs, a 40B model

$\Psi = 40 \times 10^9$ parameters, 64 GPUs of 80 GB, 8 per server

- **Memory**: $t \cdot p = 8$ fills all 80 GB $\Rightarrow$ $t \cdot p \geq 16$
- **Links**: t must stay inside a server $\Rightarrow$ $t = 8$
- **Bubble**: smallest pipeline that fits $\Rightarrow$ $p = 2$
- **The rest** is data parallel $\Rightarrow$ $d = 64 \,/\, 16 = 4$

Result

Layout **(d, t, p) = (4, 8, 2)**: 40 GB of model state per GPU, tensor groups inside servers, pipeline across.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021.

Check yourself

Two minutes with a neighbour.

- $\Psi = 13 \times 10^9$ on 80 GB GPUs: is $t \cdot p = 8$ enough for $16\Psi/(t \cdot p)$?
- Why is $t = 16$ a bad idea on servers of 8 GPUs?
- 64 GPUs with $t = 8$, $p = 4$: what is d?

IN-CLASS BUILD

Bubbles and 3D layouts

Work in pairs. Predict first; measure in Lab 5.

Exercise 1: a two-stage pipeline

m (microbatches)	**1**	**4**	**8**
predicted bubble			
measured bubble (Lab 5)			

SETUP

Deeper MLP; stage 0 = embedding + hidden 1–4, stage 1 = output + loss; batch 64; GPipe.

STRETCH

Message size per microbatch at m = 4? Bandwidth- or latency-bound?

Starter: stage 1 of a GPipe step

```
for y in ys:                          # m forwards
    h = torch.empty(b, 64)
    dist.recv(h, src=0)
    h.requires_grad_()
    loss = F.cross_entropy(stage1(h), y) / m
    saved.append((h, loss))
for h, loss in saved:                 # m backwards
    loss.backward(); dist.send(h.grad, dst=0)
```

- Divide the loss by **m**: the summed gradient is the batch mean.
- Stage 0 mirrors it: send each `h`, receive each gradient.

Check

Same order on both stages → blocking send/recv **cannot deadlock**.

Measuring the bubble

$$\text{measured bubble} = 1 - \text{busy} \;/\; \text{wall}$$

- **busy**: time a stage spends in its own forward and backward.
- **wall**: time of the whole step (median of many steps).
- Compare with the predicted $(p - 1)/(m + p - 1)$.

Watch out

Expect measured $>$ predicted: per-microbatch **overheads** do not shrink with b.

Exercise 1: solutions

m (microbatches)	**1**	**4**	**8**
predicted bubble	50%	20%	11%

Watch out

Stretch: at m = 4 a message is **4 KB**: latency-bound. For our tiny MLP a deeper pipeline does **not** pay off.

In our pipeline

Lab 5: report predicted vs measured bubble and **explain the gap**.

Exercise 2: choose a 3D layout

(d, t, p)	GB per GPU	t in one server?	bubble, m = 16
A. (16, 4, 1)			
B. (8, 8, 1)			
C. (4, 8, 2)			

GIVEN

$\Psi = 20 \times 10^9$; 64 GPUs, 8 per server; at most 60 GB of model state per GPU. **Which layout would you pick, and why?**

Exercise 2: solutions

(d, t, p)	GB per GPU	t in one server?	bubble, m = 16
A. (16, 4, 1)	80 — too big	yes	0%
B. (8, 8, 1)	40	yes	0%
C. (4, 8, 2)	20	yes	6%

Answer

Pick **B**: t · p just large enough to fit, tensor groups inside a server, **no bubble**. Give the rest to data parallelism.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. Takeaway #2; formulas from Part 3.

Key takeaways

- Pipeline parallelism cuts the model **by layers**; only the activation at a cut crosses GPUs.
- Balance stages by **work**: the pipeline runs at the pace of the slowest stage.
- Microbatches fill the pipeline: bubble $\mathbf{(p-1)/(m+p-1)}$, so $m \gg p$.
- **1F1B** keeps the bubble and stores only $\mathbf{p}$ microbatches (GPipe: m).
- $\mathbf{N = d \cdot t \cdot p}$: t inside a server, p across, d outermost; memory $\approx \mathbf{16\Psi/(t \cdot p)}$.

Next time, and readings

When	What
After this lecture	T3 checkpoint: correct DP update and basic layout tests
After this lecture	Lab 5 due: tensor and pipeline partitions, bubbles, 3D layout
Lecture 2 (this week)	Diagnosing: profiles, stragglers, MFU, energy, checkpoints

READING

Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021 (§2–3).

Huang et al., "GPipe," NeurIPS 2019.